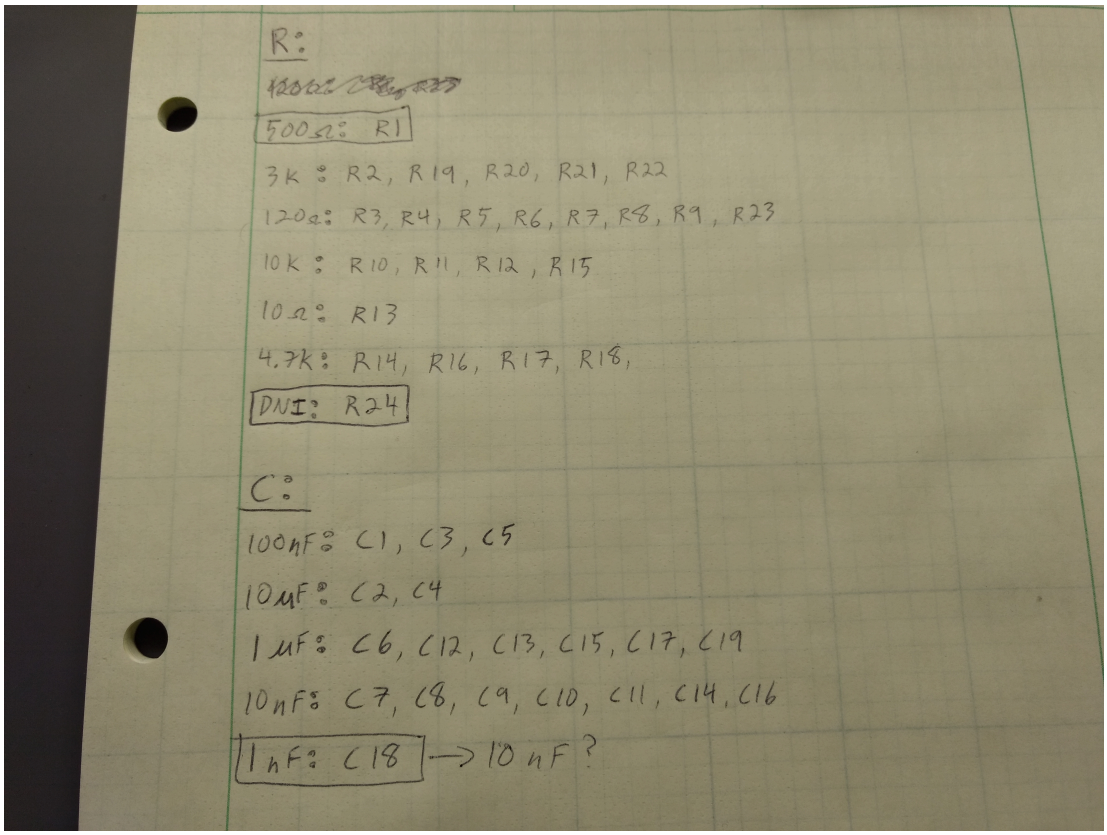
How to: Assemble & Program B-Sides FPGA Board

▼ Board Assembly

- [ ] Solder resistors and capacitors to SM PCB.

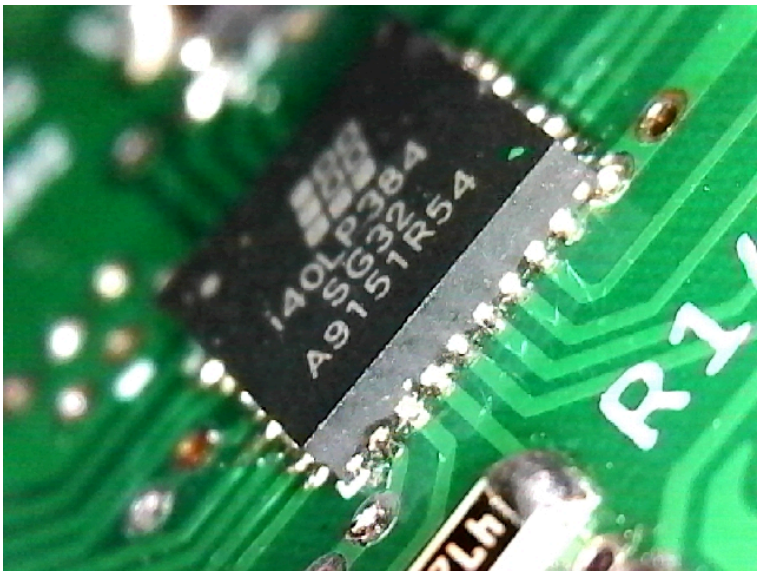
▼ Resistor & Capacitor Values



- [ ] Solder remaining components, except 7SD and FPGA.
- [ ] Solder 7SD.

- [ ] Test 3.3 V rail.
- [ ] Solder FPGA.
  - [ ] Carefully align FPGA on board, and add solder ball to each pin.
  - [ ] Cover pins with solder flux.
  - [ ] Use reflow gun sparingly to reflow solder connections.
  - [ ] Check connections under microscope.

▼ Checking FPGA Solder Joints with Microscope [JPG]



▼ EEPROM Programming

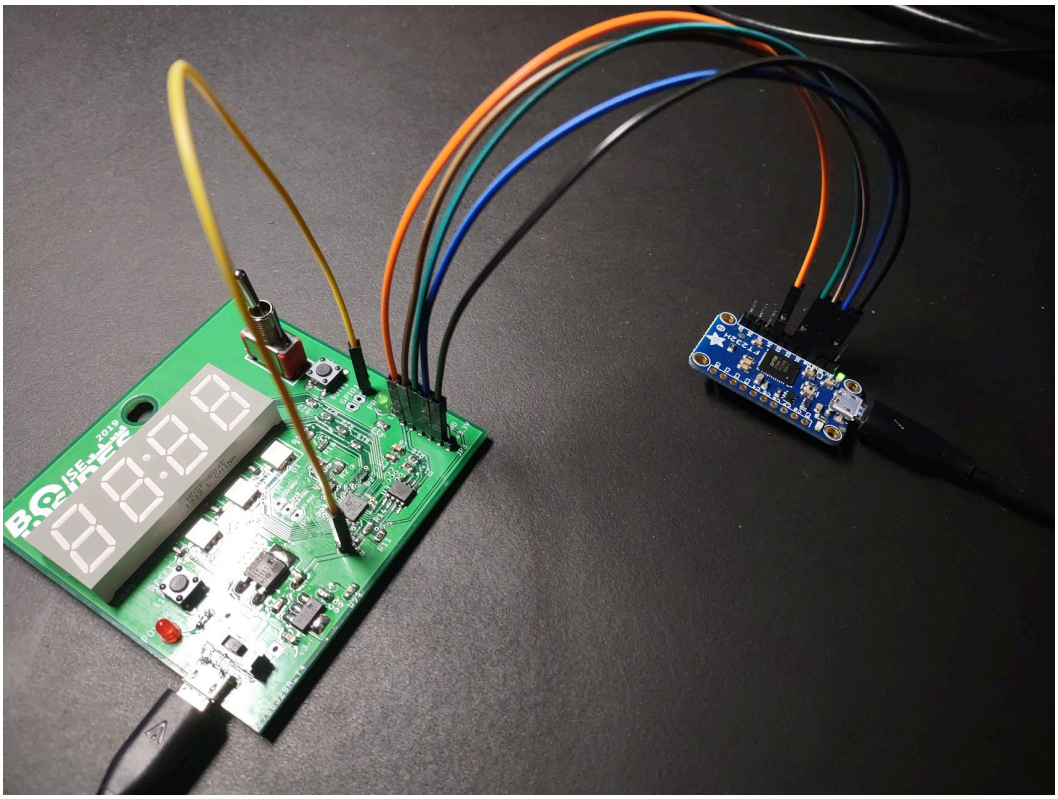
▼

Method 1: Lattice Diamond Programmer & Adafruit FT232H Breakout Board

- [ ] Flip SM power switch to BATT.
- [ ] Connect wires between SM board and Adafruit FT232H board, and connect right-most GPIO pin to TP1, as shown in the following picture.

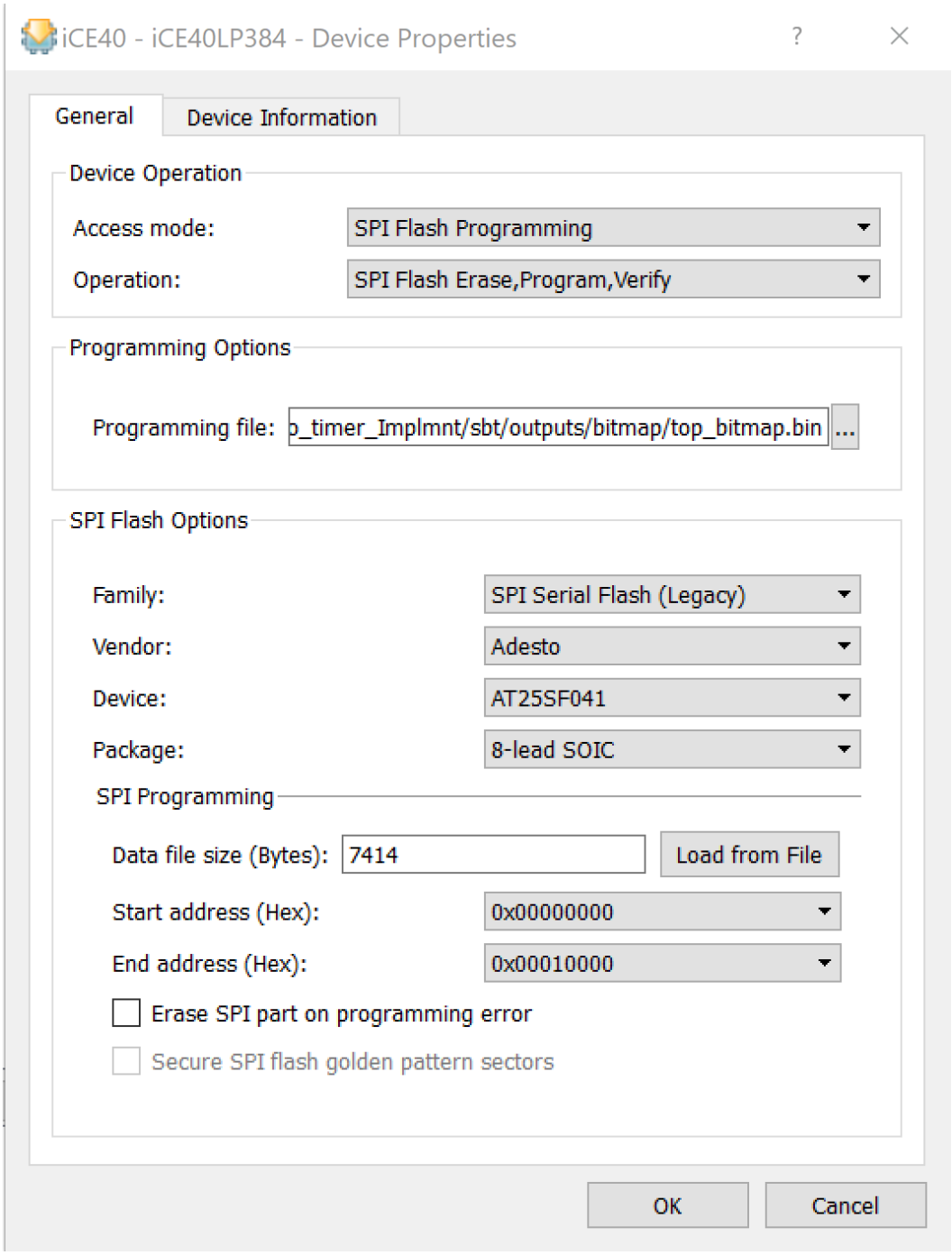
▼ Connecting SM Board to Adafruit FT232H for EEPROM Programming

[JPG]



- [ ] Connect Adafruit FT232H to PC via USB cable.
- [ ] Flip SM power switch to USB.
- [ ] Download and install FTDI VCP drivers from: VCP Drivers | FTDI.
- [ ] Open Windows Device Manager and ensure that device shows up in “Universal Serial Bus controllers” as “USB Serial Converter”. If the device (perhaps named “Single RS232-HS”) is instead listed in a category called “libusbK USB Devices”, then uninstall that device and power cycle the FT232H board.
- [ ] Download and install “Programmer Standalone X.XX 64-bit for Windows” from: Lattice Diamond Software | Lattice.
- [ ] Open Diamond Programmer, and set “Device Family” to “iCE40” and “Device” to “iCE40LP384”. Right click on that device and select “Device Properties”. In the window that pops up, select the options in the following screen snip, then select the filename for the FPGA bitfile to load.

▼ Diamond Programmer - Device Properties [snip]



- [ ] Press “Program” button in Diamond Programmer.
- [ ] Remove wires from BSides board.

▼

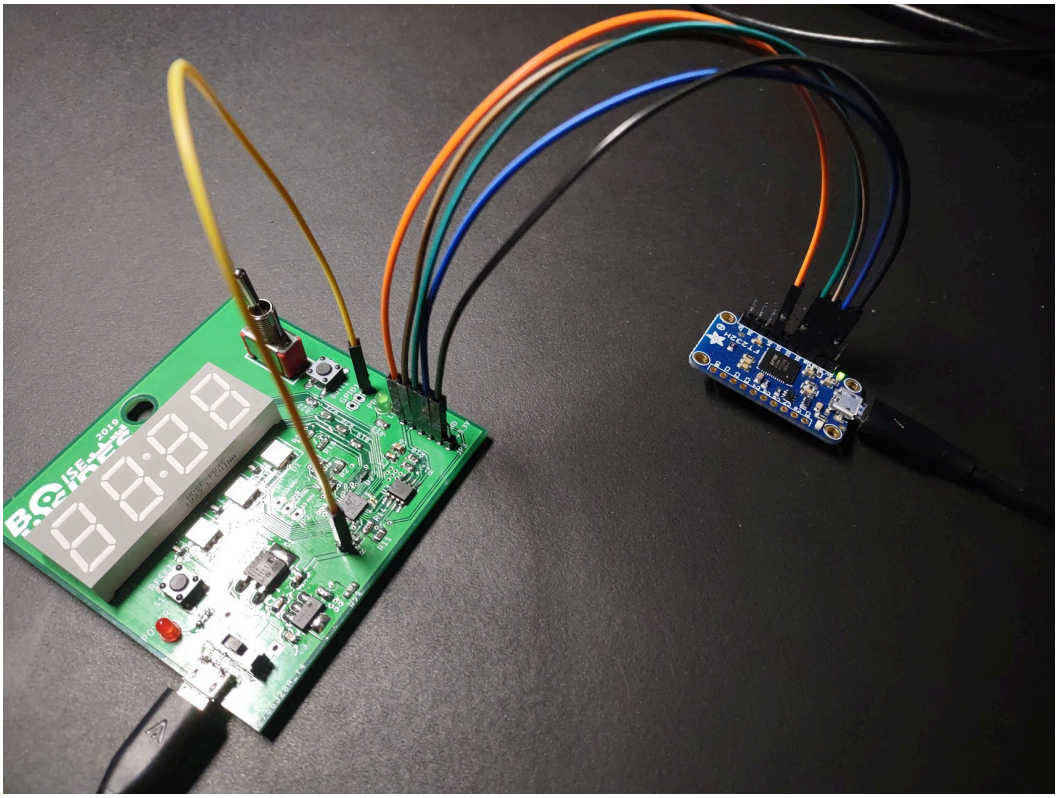
Method 2: Using Adafruit ftdiflash & Adafruit FT232H Breakout Board

- Reference: Programming SPI flash with an FT232H breakout | Adafruit

☐ Flip SM power switch to BATT.

☐ Connect wires between SM board and Adafruit FT232H board, and connect right-most GPIO pin to TP1, as shown in the following picture.

▼ Connecting SM Board to Adafruit FT232H for EEPROM Programming [JPG]



☐ Connect Adafruit FT232H to PC via USB cable.

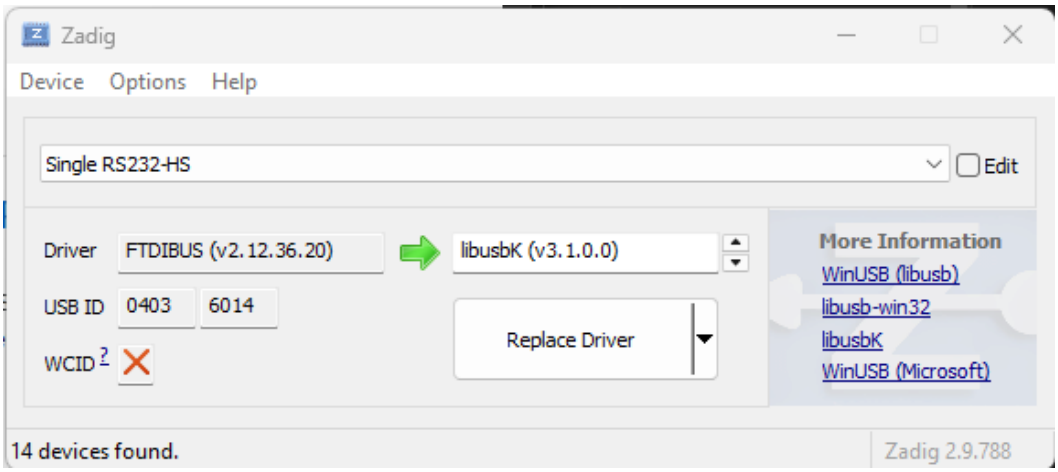
☐ Flip SM power switch to USB.

☐ Download and install Zadig tool from: Zadig.

☐ Open Zadig, and go to “Options” | “List All Devices”.

☐ Select “USB Serial Converter” device. (Note: device could instead be listed as “Single RS232-HS” for some reason.) Change Driver from “FTDIBUS” to “libusbK”. Select “Replace Driver”.

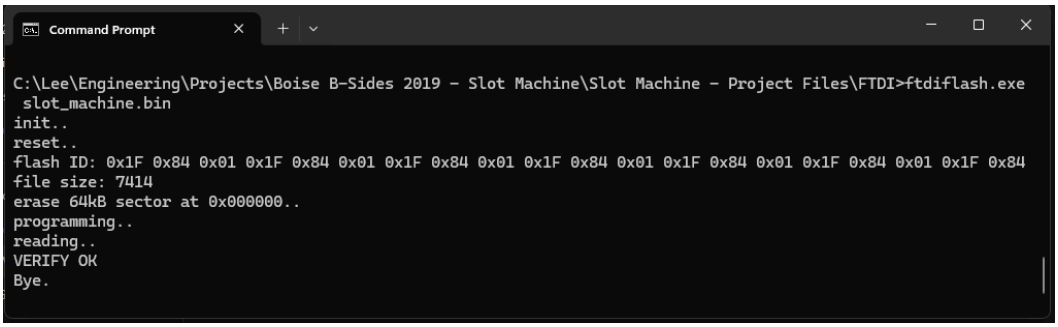
▼ Replacing FTDI driver using Zadig [snip]



☐ Download ftdiflash from: ftdiflash | GitHub | Adafruit.

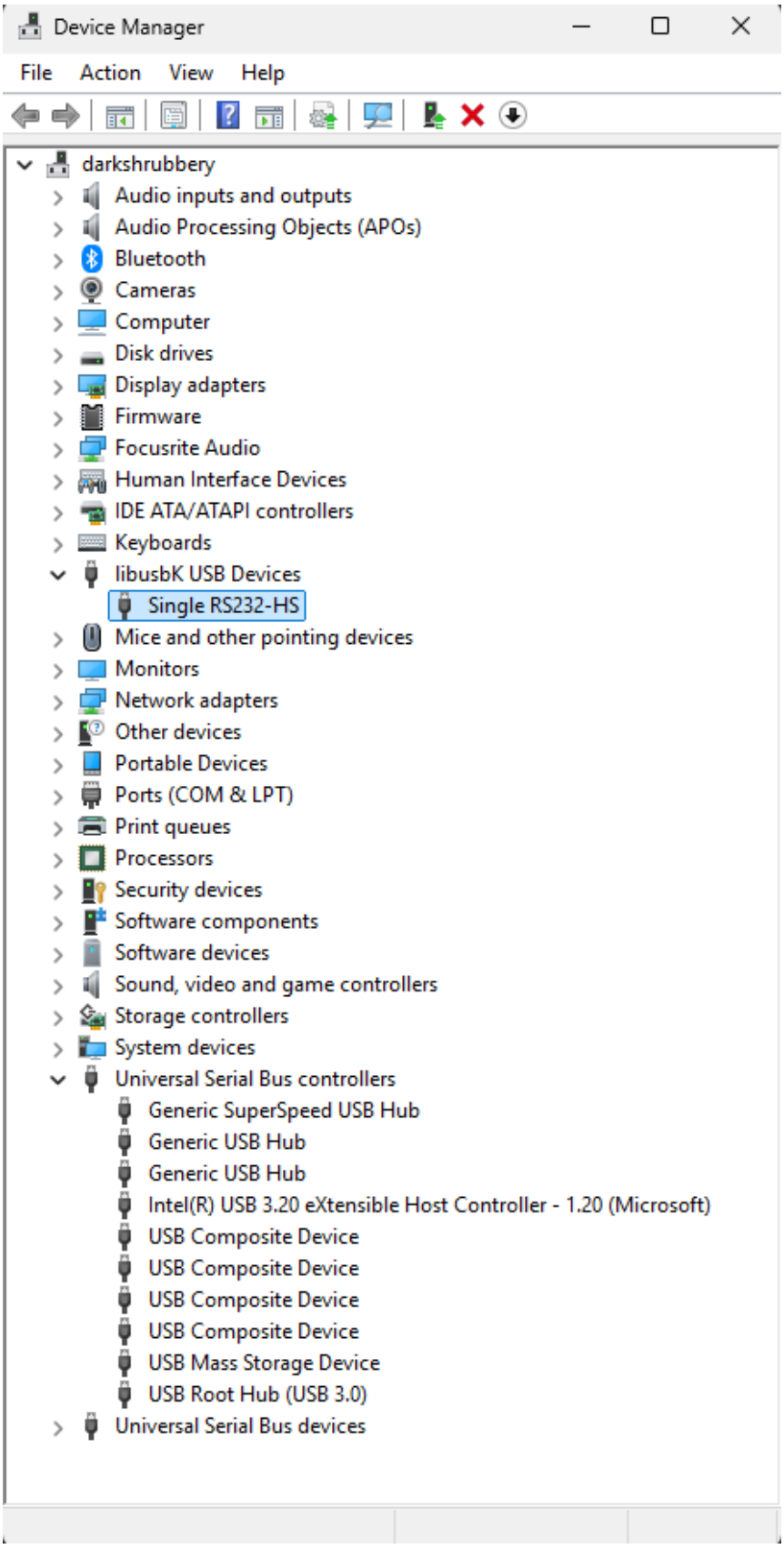
☐ Open Command Prompt to ftdiflash directory, and run: “ftdiflash.exe <binfile_directory/binfile_name>.bin”

▼ Programming the EEPROM with ftdiflash [snip]



☐ Uninstall libusbK device driver from Device Manager to re-enable programming with Diamond Programmer method.

▼ Uninstall libusbK Device Driver [snip]


Device Manager
File
Action
View
Help
darkshrubbery
Audio inputs and outputs
Audio Processing Objects (APOs)
Bluetooth
Cameras
Computer
Disk drives
Display adapters
Firmware
Focusrite Audio
Human Interface Devices
IDE ATA/ATAPI controllers
Keyboards
libusbK USB Devices
Single RS232-HS
Mice and other pointing devices
Monitors
Network adapters
Other devices
Portable Devices
Ports (COM & LPT)
Print queues
Processors
Security devices
Software components
Software devices
Sound, video and game controllers
Storage controllers
System devices
Universal Serial Bus controllers
Generic SuperSpeed USB Hub
Generic USB Hub
Generic USB Hub
Intel(R) USB 3.20 eXtensible Host Controller - 1.20 (Microsoft)
USB Composite Device
USB Composite Device
USB Composite Device
USB Composite Device
USB Mass Storage Device
USB Root Hub (USB 3.0)
Universal Serial Bus devices

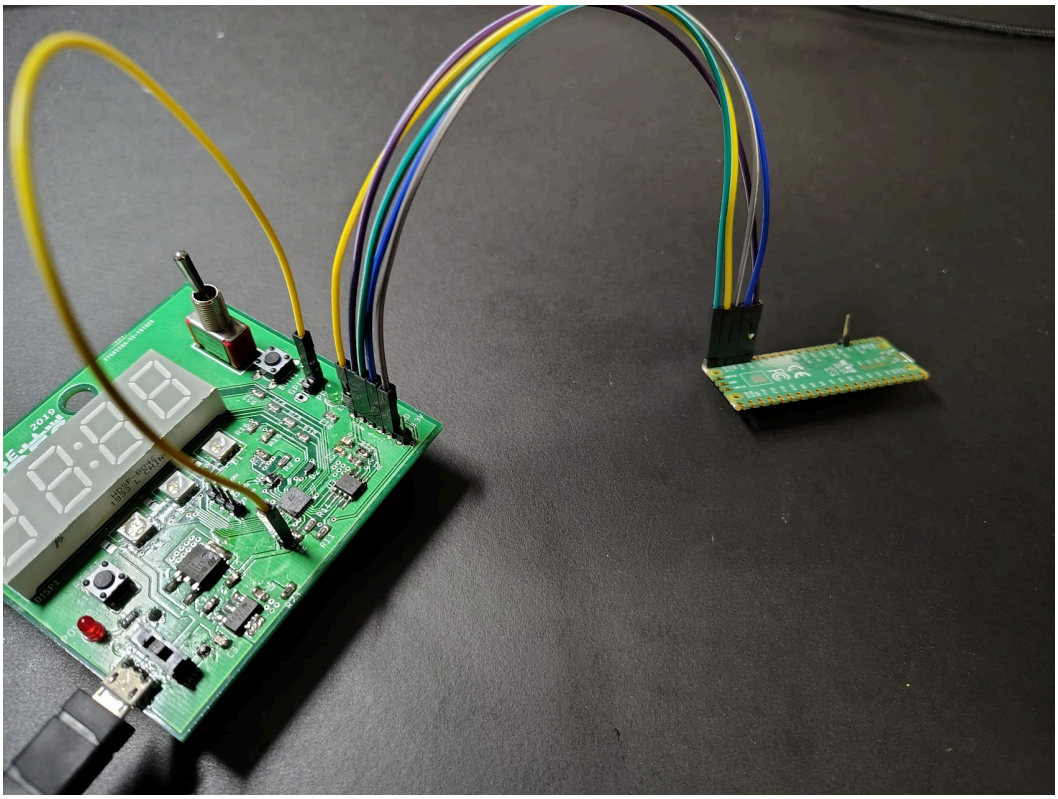
▼ Method 3: Using MicroPython & Raspberry Pi Pico

- Reference: CircuitPython Libraries on MicroPython using the Raspberry Pi Pico | Adafruit

☐ Flip SM power switch to BATT.

☐ Connect wires between SM board and Raspberry Pi Pico, and connect right-most GPIO pin to TP1, as shown in the following picture.

▼ Programming EEPROM Using Raspberry Pi Pico [JPG]



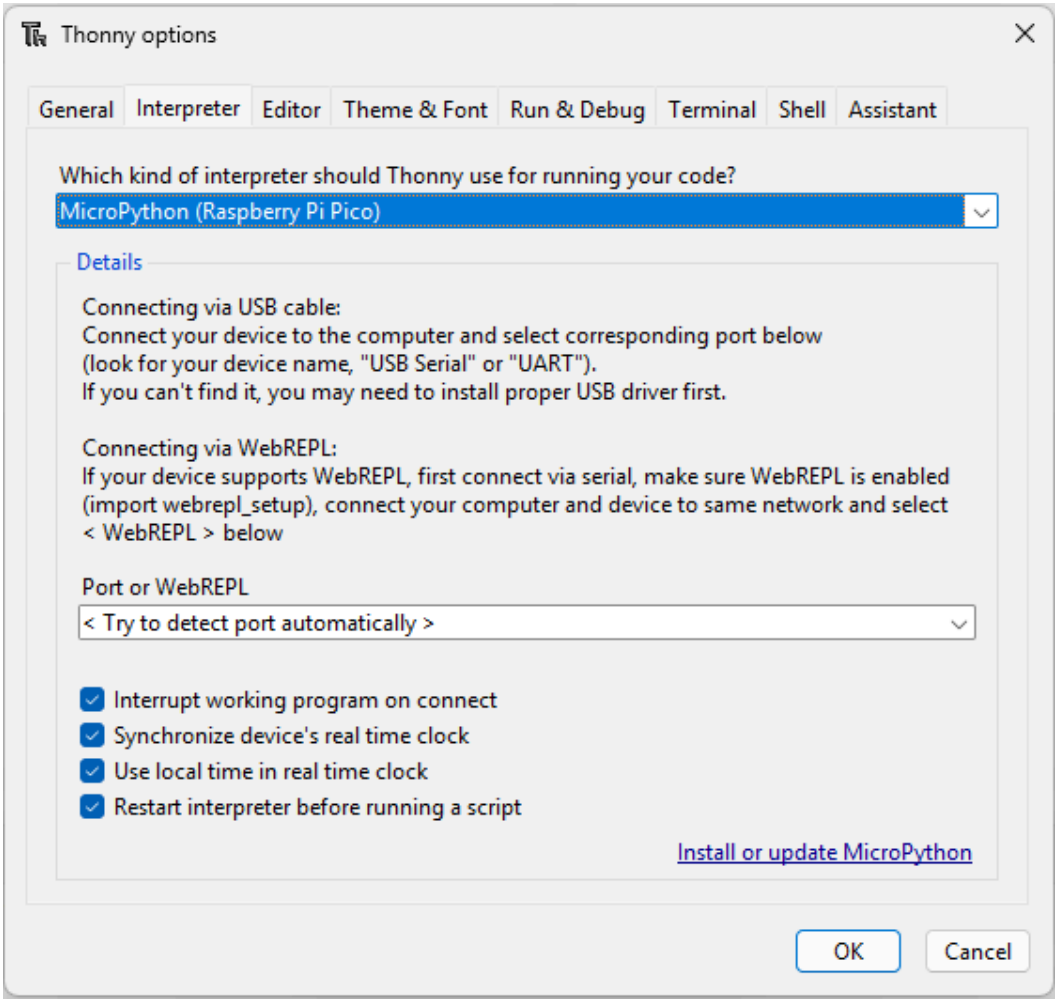
☐ Connect Pico to PC via USB cable.

☐ Flip SM power switch to USB.

☐ Download and install Thonny from: Thonny.

☐ Open Thonny, and go to “Run | Configure interpreter”. Select “MicroPython (Raspberry Pi Pico)” as the interpreter and “Try to detect port automatically” for Port. Press “OK”.

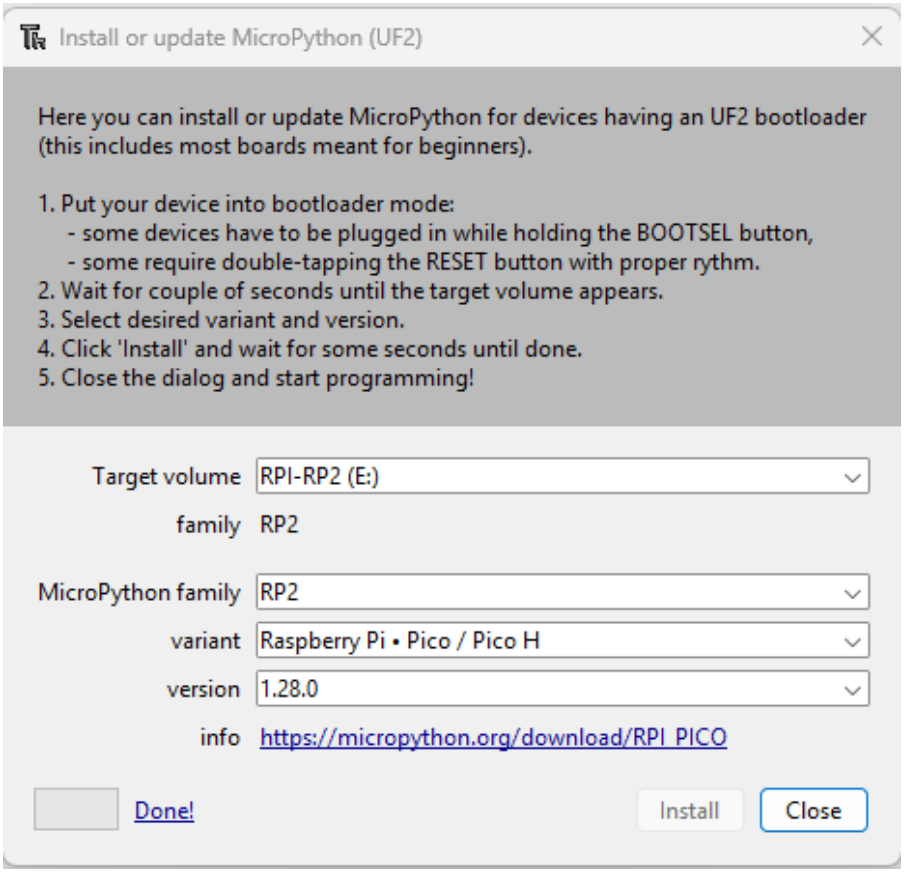
▼ Thonny Interpreter Options [snip]



☐ Press and hold the “BOOTSEL” button on the Pico, then power cycle the Pico at the PC USB port. Release the “BOOTSEL” button.

☐ Reopen the “Run | Configure interpreter” window, and select “Install or update MicroPython”. Make the selections per the following snip:

▼ Install MicroPython from Thonny [snip]



- [ ] Ensure that “Files” in the “View” menu is selected.
- [ ] Transfer bitfiles (.bin format only) to Pico.
- [ ] Create a new text file. Copy the following code into the text file, and save it to the Pico.

▼ Program Code: flash_programmer.py [generated by ChatGPT]

```
from machine import Pin, SPI
import time

# ----------------------------
# SPI + Pin Setup
# ----------------------------
spi = SPI(0,
```

```
            baudrate=5_000_000,
            polarity=0,
            phase=0,
            sck=Pin(18),
            mosi=Pin(19),
            miso=Pin(16))

cs = Pin(17, Pin.OUT)
cs.value(1)

# Optional: FPGA reset control
reset = Pin(20, Pin.OUT)
reset.value(1)

# ----------------------------
# Low-level helpers
# ----------------------------
def select():
    cs.value(0)

def deselect():
    cs.value(1)

def write_cmd(cmd, data=b''):
    select()
    spi.write(bytes([cmd]) + data)
    deselect()

def read_cmd(cmd, length):
    select()
    spi.write(bytes([cmd]))
    result = spi.read(length)
    deselect()
    return result

# ----------------------------
```

```
# Flash commands
# ----------------------------
def read_jedec():
    return read_cmd(0x9F, 3)
#def read_jedec():
#    cs.value(0)
#    time.sleep_ms(10)
#    spi.write(bytes([0x9F]))
#    result = spi.read(3)
#    cs.value(1)
#    return result

def write_enable():
    write_cmd(0x06)

def read_status():
    return read_cmd(0x05, 1)[0]

def wait_ready():
    while read_status() & 0x01:
        time.sleep_ms(1)

def chip_erase():
    print("Erasing chip...")
    write_enable()
    write_cmd(0xC7)
    wait_ready()
    print("Erase done")

def page_program(addr, data):
    write_enable()
    cmd = bytes([
        0x02,
        (addr >> 16) & 0xFF,
        (addr >> 8) & 0xFF,
        addr & 0xFF
```

```
    ])
    select()
    spi.write(cmd + data)
    deselect()
    wait_ready()

def read_data(addr, length):
    cmd = bytes([
        0x03,
        (addr >> 16) & 0xFF,
        (addr >> 8) & 0xFF,
        addr & 0xFF
    ])
    select()
    spi.write(cmd)
    data = spi.read(length)
    deselect()
    return data

# ----------------------------
# Programming function
# ----------------------------
def program_flash(filename):
    # Hold FPGA in reset (important!)
    reset.value(0)
    time.sleep_ms(10)

    # Load file
    with open(filename, "rb") as f:
        data = f.read()

    print("File:", filename)
    print("Size:", len(data), "bytes")

    # Erase
    chip_erase()
```

```
    # Program
    print("Programming...")
    for addr in range(0, len(data), 256):
        chunk = data[addr:addr+256]
        page_program(addr, chunk)

        if addr % 4096 == 0:
            print("Progress:", addr, "/", len(dat
a))

    # Verify
    print("Verifying...")
    readback = read_data(0, len(data))

    if readback == data:
        print("✅ SUCCESS")
    else:
        print("❌ VERIFY FAILED")

    # Release FPGA → boots from flash
    print("Releasing FPGA...")
    reset.value(1)

# ----------------------------
# Main menu
# ----------------------------
print("JEDEC ID:", [hex(x) for x in read_jedec()])

print("\nSelect image:")
print("1: pomo_timer.bin")
print("2: slot_machine.bin")

choice = input("Enter 1 or 2: ")

if choice == "1":
```

```
    program_flash("pomo_timer.bin")
elif choice == "2":
    program_flash("slot_machine.bin")
else:
    print("Invalid selection")
```

☐ Run the “flash_programmer.py” program on the Pico by pressing the “Run current script” button.

▼ EEPROM Successfully Programmed [snip]

